Testing Report

PREPARED BY
MUZA
MAX
MAHMOUD HASAN

INDEX

1 Introduction 3
1.1 Testing of the Port Scanning Tool 3
1.2 Ethical Considerations 3
2 Testing Environment 4
3 Testing Methodology 4
4 Testing Cases 6
4.1 The Menu 6
4.2 TCP Scan Test 7
4.3 UDP Scan Test 8
4.4 Saving Results 10
5 Edge Cases 11
6 Observations 12
7 Conclusion 12

1 Introduction

1.1 Testing of the Port Scanning Tool

Testing is an essential phase in the development of any network analysis tool. In the context of this project, testing was conducted to verify that the implemented port scanning techniques correctly identify the state of network ports and behave as expected under different conditions.

The objective of the testing phase was to evaluate whether the scanner could reliably detect **open**, **closed**, and **filtered** ports on a target system. This was achieved by running the tool against a controlled target environment and comparing the results with those obtained from established scanning tools such as **Nmap**.

The testing process also allowed verification of several important aspects of the scanner, including:

- Correct detection of **open TCP ports**
- Identification of **closed ports through TCP reset responses**
- Handling of **UDP ports with and without responses**
- Proper management of **timeouts and retries**
- Observation of the generated network traffic using **Wireshark**

By performing these tests, it was possible to confirm that the implemented scanning logic behaved according to the expected networking principles and produced results consistent with standard port scanning tools

1.2 Ethical Considerations

All experiments conducted in this project were performed in a controlled laboratory environment. The scans were executed only on virtual machines specifically created for testing purposes, including the intentionally vulnerable system **Metasploitable 2**.

The testing environment was isolated using **VMware Workstation**, ensuring that no external networks or unauthorized systems were affected. The purpose of the scans was strictly educational, aiming to understand the behaviour of network services and port scanning techniques while respecting ethical cybersecurity practices.

2 Testing Environment

The port scanning tool was tested in a controlled virtual laboratory environment composed of one host machine and two virtual machines. The goal was to simulate a small network in which the scanner could analyse a target system running vulnerable services.

The testing environment consisted of the following systems:

System	Description	IP Address
Host Laptop	Machine used to execute the scanning tool	192.168.130.1
Kali Linux VM	Running Wireshark	192.168.163.128
Metasploitable2 VM	Target machine	192.168.163.130

The penetration testing distribution used in the experiment was **Kali Linux**.

The target system was **Metasploitable 2**, a deliberately vulnerable virtual machine commonly used for security testing and penetration-testing practice.

All machines were connected within a virtual network created using **VMware Workstation**, allowing the scanner to communicate directly with the target system through the local network.

Network traffic during the scans was also analysed using **Wireshark** in order to observe packet behaviour such as TCP handshakes, UDP probes, and ICMP responses.

3 Testing Methodology

Several tests were conducted to evaluate the accuracy and reliability of the implemented port scanning tool. The goal of the testing phase was to verify that the scanner correctly identifies the state of network ports and behaves consistently under different network conditions.

The testing methodology focused on evaluating the tool across multiple scenarios, including scanning known open ports, detecting closed ports, handling filtered or unresponsive ports, and verifying timeout and retry mechanisms. The results obtained from the scanner were also compared with those produced by the industry-standard scanning tool **Nmap** in order to validate the correctness of the implementation.

1. **Scanning Known Open Ports**

To verify that the scanner correctly detects open ports, tests were performed against services known to be running on the target machine. The intentionally vulnerable system **Metasploitable 2** was used as the target because it contains several active network services.

Ports such as FTP (21), SSH (22), and HTTP (80) were scanned. These ports were expected to be open, and the scanner successfully identified them as active services. Observing these results confirmed that the tool could detect open ports through the responses returned by the target system.

2. **Scanning Known Closed Ports**

The scanner was also tested against ports that were known to have no running services. When scanning closed ports using TCP, the target system responded with TCP reset (RST) packets, indicating that the port was closed.

The scanner correctly interpreted these responses and classified the ports as closed. This test confirmed that the tool properly interprets negative responses returned by the target system.

3. Scanning Filtered or Non-Responsive Ports

Another important test involved scanning ports that did not return any response. This situation may occur when a firewall filters incoming packets or when a UDP service simply ignores the probe.

In these cases, the scanner reported the ports as **filtered**, which is consistent with the behaviour of professional scanning tools such as **Nmap**. This classification indicates that the scanner did not receive a response and therefore cannot definitively determine whether the port is open or blocked by a filtering device.

4. TCP Scan Testing

TCP scanning was performed to verify the correct detection of port states through the analysis of TCP responses. During these tests, the scanner sent TCP connection attempts to various ports on the target system.

The results were analysed to confirm that the scanner correctly interpreted different TCP responses, including successful connections to open ports and reset responses from closed ports. Network traffic was also observed using **Wireshark** to verify the presence of expected TCP handshake patterns.

5. UDP Scan Testing

UDP scanning was tested by sending UDP probes to various ports on the target system. Because UDP is a connectionless protocol, many services do not respond to unexpected packets.

When a UDP port was closed, the target system returned an ICMP "Port Unreachable" message. When no response was received after several retries, the scanner classified the port as **filtered**. These results reflect the typical behaviour of UDP scanning techniques used by professional tools.

6. Retry Mechanism

UDP communication is inherently unreliable, and packets may be lost or ignored. To address this, the scanner implemented a retry mechanism that sends multiple probes to the same port before determining the final port state.

Testing confirmed that the retry mechanism improved the reliability of UDP scanning results by reducing the chances of misclassifying ports due to temporary network issues.

7. Timeout Handling

The scanner also implements a timeout mechanism to avoid waiting indefinitely for responses from the target system. Tests were performed using different timeout values to ensure that the scanner could handle slow or unresponsive services without significantly delaying the scanning process.

Appropriate timeout values were selected to balance scan speed and accuracy.

8. Validation of Results

Finally, the results produced by the scanner were compared with those obtained using Nmap. This comparison allowed verification that the implemented scanning logic produced results consistent with an established and widely used network scanning tool.

4 Testing Cases

4.1 The Menu

The Testing starts with this Menu:

```
NETWORK
PORT SCAN

[1] Start Scan
[2] Export to file results
[3] Ping target and os comands for reconnaissance
[4] Quit

Main Menu :
==========================================================================================
1. Start scan
2. Export to file results
3. Ping target and os commands for reconnaissance
4. Exit
```

After clicking on Start scan via number 1, you will be asked to enter the IP you want to scan, in my case it was the IP 192.168.163.130, after that you will be prompted to choose the type of the scan, which will give you 3 options to choose from:

```
Scan type:
1 - TCP
2 - UDP
3 - TCP + UDP
```

After choosing the scan type, you may choose a single port, or a port range

```
Port option:
1 - Single port
2 - Port range
```

Which will ultimately reach us to the final prompt the way you want to display the result

```
Display mode:
1 - Show open + filtered
2 - Show open + filtered + closed
3 - Show only fully open
```

g of that functionality was successful and it showed the results as intended for each case, for example this is a test result for choosing TCP – Port Range – Show open + filtered + closed:

```
16      tcp     closed
17      tcp     closed          qotd
18      tcp     closed
19      tcp     closed          chargen
20      tcp     closed          ftp-data
21      tcp     open            ftp
22      tcp     open            ssh
23      tcp     open            telnet
24      tcp     closed
25      tcp     open            smtp
26      tcp     closed
27      tcp     closed
28      tcp     closed
29      tcp     closed
```

The following section will talk about each case of the TCP and UDP scan.

4.2 TCP Scan Test

The scanner was executed against the target host (192.168.163.130)

for the first 100 TCP ports.

The tool successfully detected several open services as it shows in the next image:

```
Scanning...


PORT      PROTO    STATE          SERVICE
-----------------------------------------
21        tcp      open           ftp
22        tcp      open           ssh
23        tcp      open           telnet
25        tcp      open           smtp
53        tcp      open           domain
80        tcp      open           http
```

Also that can be confirmed by the Wireshark readings [SYN, ACK] as well as the nmap findings:

```
┌──(kali㉿kali)-[~]
└─$ nmap -p 1-100 192.168.163.130
Starting Nmap 7.98 ( https://nmap.
Nmap scan report for 192.168.163.1
Host is up (0.0029s latency).
Not shown: 94 closed tcp ports (re
PORT   STATE SERVICE
21/tcp open  ftp
22/tcp open  ssh
23/tcp open  telnet
25/tcp open  smtp
53/tcp open  domain
80/tcp open  http
MAC Address: 00:0C:29:BC:3B:DD (VM
```

192.168.163.130	192.168.163.1	TCP	66	21 → 61548 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 M
192.168.163.130	192.168.163.1	TCP	66	22 → 61553 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 M
192.168.163.130	192.168.163.1	TCP	60	23 → 61560 [ACK] Seq=1 Ack=2 Win=5856 Len=0
192.168.163.130	192.168.163.1	TCP	60	23 → 61560 [FIN, ACK] Seq=13 Ack=2 Win=5856 Len=0
192.168.163.130	192.168.163.1	TCP	66	23 → 61560 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 M
192.168.163.130	192.168.163.1	TCP	60	24 → 61561 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.163.130	192.168.163.1	TCP	60	24 → 61561 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.163.130	192.168.163.1	TCP	60	24 → 61561 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.163.130	192.168.163.1	TCP	60	24 → 61561 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
192.168.163.130	192.168.163.1	TCP	60	25 → 61549 [ACK] Seq=1 Ack=2 Win=5856 Len=0
192.168.163.130	192.168.163.1	TCP	60	25 → 61549 [FIN, ACK] Seq=56 Ack=2 Win=5856 Len=0
192.168.163.130	192.168.163.1	TCP	66	25 → 61549 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 M

4.3 UDP Scan Test

UDP scanning was performed on several commonly used service ports to evaluate the scanner's ability to detect active UDP services. Unlike TCP, UDP does not use a connection handshake. As a result, detecting the state of a UDP port is more difficult and often depends on whether the service sends a response to a probe packet.

During testing, UDP probes were sent to selected ports using protocol-specific payloads when available. For example, port **53** responded to the probe, indicating that the DNS service was active on the target machine.

For many other ports, no response was received after multiple retries. In these cases, the scanner classified the ports as **filtered**, meaning that the port could either be open but silent or blocked by a firewall.

Because the implementation relies on the Python **socket** module, the scanner does not capture ICMP error messages generated by the target host. As a result, UDP ports that are closed may also appear as filtered when no response is received.

UDP Probe-Based Detection

In this implementation, UDP scanning relies on **service-specific probe packets**. Unlike TCP, UDP services often do not respond to arbitrary or empty packets. Therefore, the scanner sends protocol-specific probes designed to trigger a response from known services.

The UDP detection mechanism currently supports a limited set of services for which probes have been implemented. These include common UDP services such as DNS, TFTP, RPCBind, NTP, NetBIOS Name Service, SNMP, SSDP, ISAKMP, RIP, and NFS.

For example:

- Port **53** uses a DNS query probe.
- Port **69** uses a TFTP read request probe.
- Port **123** uses a Network Time Protocol request.
- Port **161** uses an SNMP query.

If a service receives the correct probe, it typically responds with a valid UDP reply. When such a response is detected, the scanner classifies the port as **open**.

However, if a port is scanned for which no corresponding probe exists, the scanner sends a minimal or generic UDP packet. Many UDP services ignore such packets and do not generate a response. In these cases, the scanner cannot reliably determine the port state and therefore classifies the port as **filtered**.

Because of this design, reliable UDP detection in this implementation is primarily limited to ports for which a valid probe has been implemented.

We can see the Results of the scan by showing the only open ports in the following image:

```
PORT      PROTO    STATE          SERVICE
-----------------------------------------
53        udp      open           domain
69        udp      open           tftp
```

As well as a scan from nmap which confirms the claim, note that we didn’t have all the probes database the same way that nmap has, but for those who are tested, it confirms it.

```
Not shown: 194 closed udp ports
PORT     STATE         SERVICE
53/udp   open          domain
68/udp   open|filtered dhcpc
69/udp   open|filtered tftp
```

4.4 Saving Results

After the scan ends you will be prompted to save the results of your scan in a txt file as it shows in the following image:

```
Do you want to save this scan to a file? (y/n):
```

After Pressing y the results will be saved, and you will be prompted to enter **Enter** to go to the main menu as it shows here:

```
Do you want to save this scan to a file? (y/n): y
Scan saved to scan_results_1.txt
Save time 0.09 seconds
Press Enter to return to main menu...
```

The file will contain the **IP** of the targeted device, the **type** of the Protocol **(TCP/UDP)** and the **range/IP** used in the scan as it shows in the next image.

```
Scan results for host: 192.168.163.130
Scan type: tcp
Ports: 1-100

Port status:
PORT     PROTO    STATE          SERVICE
-----------------------------------------
1        tcp      closed
2        tcp      closed
3        tcp      closed
4        tcp      closed
5        tcp      closed
6        tcp      closed
7        tcp      closed         echo
8        tcp      closed
9        tcp      closed         discard
```

5 Edge Cases

Definition:
In software testing, **edge cases** are unusual or extreme situations that may not occur frequently but can reveal limitations or unexpected behaviour in a system. For a port scanner, edge cases are network conditions, service behaviours, or environment configurations that can affect scan accuracy or reliability.

1. No Response from UDP Services

- Many UDP services do not respond to arbitrary or invalid probes.
- For example, sending a random packet to a UDP service like RPC or NFS often yields no reply.
- In your scanner, when no response is received after multiple retries, the port is classified as **filtered**.
- This is an edge case because **the port may be closed**, but the scanner cannot determine this due to the lack of ICMP capture.

2. Firewall Filtering

- Firewalls may silently drop packets without sending any reply.
- This is similar to the “no response” case, but the cause is different: network security mechanisms rather than the service itself.
- In such situations, your scanner **cannot distinguish between an open-but-silent service and a blocked port**, so it again classifies the port as **filtered**.
- This highlights the **limitation of socket-based UDP scanning**, which cannot directly detect blocked traffic.

3. Network Timeout

- If the timeout set in the scanner is too short, legitimate responses may arrive after the scanner stops waiting.
- This can result in **false negatives** (ports incorrectly classified as filtered).
- Increasing the timeout improves reliability but also increases scan duration.
- Timeouts are important to test because network latency varies, especially in virtual labs or real networks.

4. Other Edge Cases

- **Rate-limited ICMP responses**: Some systems limit ICMP error messages, causing closed ports to appear filtered.
- **Service-specific quirks**: Some services respond only to well-formed probes (DNS, NTP, SNMP), and others ignore malformed requests.
- **Simultaneous open/closed behaviour**: Rarely, a service may respond inconsistently if it is temporarily busy, which can confuse the scanner.

6 Observations

1. TCP Scanning

- TCP scanning is faster and more reliable because the TCP handshake provides clear signals **(SYN-ACK or RST-ACK)** about whether a port is open or closed.
- the scanner can classify ports accurately and consistently.

UDP Scanning

- UDP scanning is slower and less reliable because many services do not respond.
- The socket module cannot capture ICMP error messages, so closed UDP ports often appear as **filtered**.
- The scanner relies on **predefined probes**, so only ports with known probes can be detected as open.
- Multiple retries and proper timeout settings are required to improve detection accuracy.

7 Conclusion

The testing phase confirmed that the implemented port scanning tool is capable of accurately identifying open and closed TCP ports under a variety of network conditions. The scanner reliably interpreted TCP responses, such as SYN-ACK packets for open ports and RST-ACK packets for closed ports, demonstrating that the tool correctly handles the standard TCP handshake and port state determination. This reliability makes TCP scanning a robust and dependable method for network service detection.

UDP scanning, however, proved to be less predictable. Due to the connectionless nature of the UDP protocol, many services do not respond to unsolicited probes, and firewalls may silently drop packets without generating error messages. As a result, the scanner classified some UDP ports as **filtered**, reflecting uncertainty in the port's actual state. Despite the implementation of retries and timeout handling, the inherent limitations of UDP communication make precise detection more challenging than TCP scanning.

Future improvements to the scanner could address these limitations by incorporating more sophisticated techniques. For example, the tool could use **service-specific probes** tailored to common UDP services (such as DNS, NTP, or SNMP) to elicit responses more reliably. Additionally, implementing **raw packet analysis** to detect ICMP messages and other network signals could improve the accuracy of UDP scanning and help distinguish between filtered and truly open ports. These enhancements would increase the reliability and comprehensiveness of the tool, bringing its performance closer to professional network scanning software.